

| 项目       | 职责范围     | 数据源                                          | 文档版本             |
|----------|----------|----------------------------------------------|------------------|
| QuietMel | 仅模型训练与验证 | City of Melbourne Pedestrian Counting System | 1.0 / 2026-08-06 |

把近 3 个完整年份的小时人流记录，按“传感器 × 星期 × 季节 × 小时”聚合，输出每组的基准值、区间、梯度和质量标记。

## 1. 任务边界与最终交付

把历史数据处理为固定格式的模型文件，并完成离线验证。

### 需要完成

- 下载并整理官方历史小时人流数据。
- 按本指南规定的时间、单位和分组规则完成清洗与特征计算。
- 编写可重复执行的训练脚本，不使用 Notebook 作为唯一交付物。
- 输出 3 个规定文件，并完成字段、重复值、空值和指标检查。

### 最终目录

```
model/  
  train_crowd_profiles.py  
  requirements.txt  
  README.md  
output/  
  crowd_hourly_profiles.csv  
  model_metadata.json  
  validation_metrics.csv
```

## 2. 数据来源与原始字段

使用 City of Melbourne Open Data 的官方数据集：Pedestrian Counting System — Monthly Counts per Hour。

官方数据集

<https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-monthly-counts-per-hour/>

优先下载 Parquet；若官方只提供 CSV，也可使用 CSV。不要使用当前前端的实时接口响应代替历史训练数据。

| 原始字段        | 状态 | 处理规则              |
|-------------|----|-------------------|
| id          | 忽略 | 数据平台记录 ID，不参与建模   |
| location_id | 必需 | 传感器/地点稳定标识，按字符串读取 |

| 原始字段            | 状态 | 处理规则           |
|-----------------|----|----------------|
| sensing_date    | 必需 | 观测日期，解析为本地日期   |
| hourday         | 必需 | 小时，整数 0–23     |
| direction_1     | 忽略 | 方向 1 计数，不用于 v1 |
| direction_2     | 忽略 | 方向 2 计数，不用于 v1 |
| pedestriancount | 必需 | 该传感器在该小时的总人数   |
| sensor_name     | 可选 | 只用于人工核对，不作为主键  |
| location        | 可选 | 只用于人工核对位置      |

数据时间范围

每次训练默认选择当前年份之前的 3 个完整年份：

选择规则

start\_year = current\_year - 3  
end\_year = current\_year - 1  
例如在 2026 年执行时，使用 2023-01-01 至 2025-12-31。不要把不完整的当前年份混入训练集。

3. 固定数据约定

| 项目          | 固定值                 | 说明                                  |
|-------------|---------------------|-------------------------------------|
| 时区          | Australia/Melbourne | 日期、星期和小时均以墨尔本本地时间解释                 |
| day_of_week | 0–6                 | 0=Sunday, 1=Monday, ..., 6=Saturday |
| hour_of_day | 0–23                | 直接来源于 hourday                       |
| summer      | 12, 1, 2            | 澳大利亚季节                              |
| autumn      | 3, 4, 5             | 澳大利亚季节                              |
| winter      | 6, 7, 8             | 澳大利亚季节                              |
| spring      | 9, 10, 11           | 澳大利亚季节                              |
| 流量单位        | persons/minute      | 输出统一为每分钟平均人数（ppm）                   |

小时总数转为分钟平均值

```
observed_ppm = pedestriancount / 60
```

不要把小时总人数直接写入 baseline\_ppm。

4. 数据清洗步骤

- 1. 读取文件时把 location\_id 按字符串处理；只保留必需字段及可选核对字段。
- 2. 解析 sensing\_date；无法解析的记录删除并计数。
- 3. 把 hourday 转为整数，删除不在 0-23 范围内的记录。
- 4. 把 pedestriancount 转为数值；删除空值和负数。零值允许保留。
- 5. 按默认规则筛选最近 3 个完整年份。
- 6. 使用 sensing\_date 生成 day\_of\_week 和 season；使用 hourday 生成 hour\_of\_day。
- 7. 计算 observed\_ppm = pedestriancount / 60。
- 8. 检查重复记录。若同一 location\_id、sensing\_date、hourday 出现多行，先记录数量，再取 pedestriancount 的总和；不得静默丢弃。
- 9. 保留清洗统计，用于写入 model\_metadata.json。

**禁止项** 不得用 sensor\_name 代替 location\_id；不得把方向计数与 pedestriancount 再次相加；不得对负值取绝对值。

5. 分组与特征计算

训练数据的基础分组键固定为：

```
location_id × day_of_week × season × hour_of_day
```

每个分组使用其中全部 observed\_ppm 样本计算下列特征：

| 输出字段             | 计算方法                        | 用途              |
|------------------|-----------------------------|-----------------|
| baseline_ppm     | median(observed_ppm)        | 稳健基准预测值         |
| trimmed_mean_ppm | mean(remove bottom/top 10%) | 去掉两端各 10% 后的平均值 |

| 输出字段         | 计算方法                | 用途      |
|--------------|---------------------|---------|
| p25_ppm      | 25th percentile     | 预期低位区间  |
| p75_ppm      | 75th percentile     | 预期高位区间  |
| sample_count | count(observed_ppm) | 该组有效样本数 |

分位数使用 pandas/numpy 默认线性插值即可，但 README 中必须注明。若样本数小于 10，trimmed mean 仍可计算；切片后为空时退回普通平均值。

## 6. 梯度模型

v1 不训练复杂机器学习模型。每个小时段使用当前基准值和下一小时基准值之间的斜率表示趋势。

### 小时梯度

$\text{gradient\_ppm\_per\_hour} = \text{next\_hour\_baseline\_ppm} - \text{baseline\_ppm}$

### 小时内预测方程

$\text{predicted\_ppm}(t) = \text{baseline\_ppm} + \text{gradient\_ppm\_per\_hour} \times t$

$t = \text{minute} / 60$ ，范围为  $0 \leq t < 1$ 。例如 10:30 的  $t=0.5$ 。

## 下一小时键的计算

- hour 0–22：下一小时仍属于同一天、同一星期、同一季节。
- hour 23：下一小时为次日 hour 0；day\_of\_week 加 1，并按 7 取模。
- 跨季节边界：用实际日期推导下一小时的 season。不要假设次日季节一定相同。
- 如果下一小时基准值不可用，gradient\_ppm\_per\_hour 设为 0，并将质量标记提升为 fallback。

**实现提示** 为正确处理跨日和跨季节，建议先建立“日期样本 → 下一小时时间键”的映射，再连接到聚合结果；不要只用 hour+1。

## 7. 低样本与回退策略

| sample_count | quality_flag | 处理         |
|--------------|--------------|------------|
| $\geq 10$    | ok           | 直接使用完整分组统计 |

| sample_count | quality_flag | 处理                |
|--------------|--------------|-------------------|
| 5-9          | low_sample   | 仍输出完整分组统计，但标记样本不足 |
| < 5          | fallback     | 依次使用下面的回退序列       |

## 回退顺序

1. 同一传感器 + 同一 day\_of\_week + 同一 hour\_of\_day，忽略 season。
2. 同一传感器 + weekday/weekend + 同一 hour\_of\_day，忽略具体星期和季节。  
weekday=Monday-Friday； weekend=Saturday/Sunday。
3. 同一传感器 + 同一 hour\_of\_day，使用全部可用日期。
4. 仍无样本时保留该键，baseline/区间/梯度写 0，quality\_flag=fallback，并在 metadata 中记录数量。

回退后的数值写入输出字段，但 sample\_count 仍保留原始完整分组的样本数，以便下游知道为何发生回退。

## 8. 主输出文件：crowd\_hourly\_profiles.csv

文件编码必须为 UTF-8；列顺序、名称和枚举值必须完全一致。

```
location_id,day_of_week,season,hour_of_day,baseline_ppm,trimmed_mean_ppm,p25_ppm,p75_ppm,gradient_ppm_per_hour,sample_count,quality_flag,source_start_date,source_end_date,model_version
```

| 字段               | 类型      | 规则                          |
|------------------|---------|-----------------------------|
| location_id      | text    | 非空；来自原始 location_id         |
| day_of_week      | integer | 0-6； 0=Sunday               |
| season           | text    | summer/autumn/winter/spring |
| hour_of_day      | integer | 0-23                        |
| baseline_ppm     | decimal | 非负；最多 4 位小数                 |
| trimmed_mean_ppm | decimal | 非负；最多 4 位小数                 |
| p25_ppm          | decimal | 非负；最多 4 位小数                 |

| 字段                    | 类型      | 规则                     |
|-----------------------|---------|------------------------|
| p75_ppm               | decimal | 非负；最多 4 位小数            |
| gradient_ppm_per_hour | decimal | 可正可负；最多 4 位小数          |
| sample_count          | integer | 原始完整分组有效样本数            |
| quality_flag          | text    | ok/low_sample/fallback |
| source_start_date     | date    | YYYY-MM-DD             |
| source_end_date       | date    | YYYY-MM-DD             |
| model_version         | text    | 固定 hourly-gradient-v1  |

## 唯一键

```
| location_id + day_of_week + season + hour_of_day + model_version
```

该组合不得重复。数值字段不得出现 NaN、Infinity、-Infinity 或科学计数法。

## 示例记录

```
| 37,1,summer,8,12.7500,13.0241,9.5000,16.1000,2.3000,38,ok,2023-01-01,2025-12-31,hourly-gradient-v1
```

## 9. 元数据文件：model\_metadata.json

该文件说明训练范围、规则和清洗结果。字段可扩展，但以下字段必须存在。

```
{
  "model_version": "hourly-gradient-v1",
  "generated_at": "2026-08-06T10:00:00+10:00",
  "timezone": "Australia/Melbourne",
  "source_dataset": "pedestrian-counting-system-monthly-counts-per-hour",
  "source_url": "https://data.melbourne.vic.gov.au/explore/dataset/pedestrian-counting-system-monthly-counts-per-hour/",
  "source_start_date": "2023-01-01",
  "source_end_date": "2025-12-31",
  "input_row_count": 0,
  "clean_row_count": 0,
  "dropped_row_count": 0,
  "duplicate_group_count": 0,
  "sensor_count": 0,
  "profile_row_count": 0,
  "quality_counts": {"ok": 0, "low_sample": 0, "fallback": 0},
  "unit": "persons_per_minute",
  "day_of_week_convention": "0=Sunday,...,6=Saturday",
  "season_convention": "Australian meteorological seasons",
  "gradient_method": "next_hour_baseline_minus_current_hour_baseline"
}
```

## 10. 离线验证

验证只用于判断模型是否可接受，不改变输出字段。把时间范围内最近 4 周作为测试集，其余记录作为训练集。

1. 按 sensing\_date 排序，找到最大日期。
2. 测试集 = 最大日期往前 27 天至最大日期；训练集 = 更早的记录。
3. 只用训练集重新计算 profile。
4. 为测试集每条记录按 location\_id/day\_of\_week/season/hour\_of\_day 查找预测值，并应用相同回退规则。
5. 比较 predicted\_ppm 与 observed\_ppm，按传感器计算 MAE 和 WAPE。

### MAE

```
mae_ppm = mean(abs(observed_ppm - predicted_ppm))
```

### WAPE

```
wape_percent = sum(abs(observed_ppm - predicted_ppm)) / sum(observed_ppm) × 100
```

当 observed\_ppm 总和为 0 时，WAPE 留空并在质量标记中说明。

### validation\_metrics.csv 固定字段

```
location_id,mae_ppm,wape_percent,test_sample_count,quality_flag
```

quality\_flag 建议规则：测试样本数  $\geq 10$  为 ok，5–9 为 low\_sample， $< 5$  为 fallback。指标最多保留 4 位小数。

**说明** v1 不设强制准确率门槛。验收重点是可重复、无数据泄漏、单位正确、回退透明。若个别传感器 WAPE 异常高，应在 README 中列出并说明。

## 11. 自动检查与交付前验收

| 检查项  | 通过标准              |
|------|-------------------|
| 文件存在 | 3 个 output 文件均已生成 |
| 列契约  | 列名和列顺序完全一致        |



| 检查项  | 通过标准                                         |
|------|----------------------------------------------|
| 唯一性  | 主输出唯一键无重复                                    |
| 取值范围 | day 0–6、hour 0–23、season 与 quality_flag 枚举合法 |
| 数值质量 | 无 NaN/Infinity；除 gradient 外均非负               |
| 日期   | YYYY-MM-DD，且为 3 个完整年份                        |
| 精度   | 小数最多 4 位                                     |
| 可重复  | 相同输入连续执行两次，输出内容一致                            |
| 说明文件 | README 包含运行命令、依赖、时间范围和异常说明                   |

建议在脚本末尾执行的断言

```
assert profiles[KEY_COLUMNS].duplicated().sum() == 0
assert profiles['day_of_week'].between(0, 6).all()
assert profiles['hour_of_day'].between(0, 23).all()
assert set(profiles['season']) <= {'summer', 'autumn', 'winter', 'spring'}
assert set(profiles['quality_flag']) <= {'ok', 'low_sample', 'fallback'}
assert np.isfinite(profiles[NUMERIC_COLUMNS].to_numpy()).all()
```

12. README 必须写明的内容

- Python 版本、安装命令和完整运行命令。
- 输入文件来源、下载日期、输入格式和时间范围。
- 字段映射、时区、星期编码、季节规则和 ppm 单位。
- 中位数、10% trimmed mean、分位数和梯度的计算方法。
- 重复记录、缺失值、低样本和下一小时缺失的处理方式。
- 验证集划分方式、整体或典型 MAE/WAPE，以及已知异常。
- 输出目录和 3 个交付文件的用途。

13. 交付清单

| 状态 | 交付要求 |
|----|------|
|----|------|

| 状态                       | 交付要求                                        |
|--------------------------|---------------------------------------------|
| <input type="checkbox"/> | train_crowd_profiles.py 可在空环境按 README 运行    |
| <input type="checkbox"/> | requirements.txt 只包含实际使用的依赖                 |
| <input type="checkbox"/> | crowd_hourly_profiles.csv 通过所有自动检查          |
| <input type="checkbox"/> | model_metadata.json 数字与实际输出一致               |
| <input type="checkbox"/> | validation_metrics.csv 覆盖所有可验证的 location_id |
| <input type="checkbox"/> | README.md 已记录运行方法、规则和异常                     |
| <input type="checkbox"/> | 压缩包中不含原始密钥、Supabase key 或 .env 文件           |

**提交给项目负责人** 提交整个 model 文件夹。项目负责人将检查输出契约后创建数据库表、上传数据，并完成 Express 与前端对接。

## 附录 A：下游如何使用这些字段

| 字段                    | 后续用途          |
|-----------------------|---------------|
| baseline_ppm          | 作为当前小时预测基准    |
| gradient_ppm_per_hour | 根据当前分钟修正预测趋势  |
| p25_ppm / p75_ppm     | 前端显示正常预测区间    |
| quality_flag          | 提示低样本或回退结果    |
| model_version         | 支持后续模型版本切换与回滚 |
| source_start/end_date | 显示模型数据覆盖范围    |

后端查询时会使用当前传感器、星期、季节和小时定位一条 profile，再按当前分钟计算 predicted\_ppm。所有命名和单位之所以固定，是为了避免导入后再次转换。

## 附录 B：最终数据流

|           |         |         |          |
|-----------|---------|---------|----------|
| 1. 官方历史数据 | 2. 训练脚本 | 3. 固定输出 | 4. 项目负责人 |
|-----------|---------|---------|----------|

| 1. 官方历史数据 | 2. 训练脚本     | 3. 固定输出         | 4. 项目负责人                |
|-----------|-------------|-----------------|-------------------------|
| 小时人数记录    | 清洗、聚合、梯度、验证 | CSV + JSON + 指标 | Supabase → Express → 前端 |

END OF HANDOFF GUIDE